

Python for Data Analysts

Practical Questions Only — Coding & Data-Solving Workbook

Important: This workbook contains no theory/definition questions. Every question requires you to write code, run code, manipulate data, calculate results, clean data, or create a visualization.

Learning Roadmap

- 1. Python Fundamentals
- 2. Python Data Structures
- 3. NumPy
- 4. Pandas
- 5. Data Visualization
- 6. Exploratory Data Analysis (EDA)
- 7. Real Data Analyst Projects
- 8. Interview Preparation

1. Python Fundamentals

Variables

1. Create variables for product name, quantity, unit price, and discount percentage, then calculate the final amount.
2. Create variables for five monthly sales values and calculate total sales.
3. Store a customer's first name and last name separately, then create and print the full name.
4. Store revenue and cost in variables, then calculate profit and profit margin.
5. Store total revenue and number of orders, then calculate average order value.
6. Store an original price and discount rate, then calculate the discounted price.
7. Create three product variables in one statement and print them in a formatted line.
8. Swap the values of two sales variables and print the values before and after the swap.
9. Store salary and bonus percentage, then calculate total compensation.
10. Use variables for quantity, price, tax rate, and shipping cost to calculate a final bill.

Data Types

1. Create variables for customer ID, name, order amount, and order status; print each value and its type.
2. Convert the string ``1500`` to a number and calculate a 10% discount.
3. Convert the number ``2500`` to a string and combine it with ``Revenue: ``.
4. Create a Boolean that indicates whether sales are above 10000 and print it.
5. Create a list of five prices and print its type.
6. Create a dictionary containing customer name, age, and purchase amount, then print its values.
7. Create variables using integer, float, string, Boolean, list, and dictionary values and print their types.
8. Convert a quantity stored as text into an integer and calculate revenue.
9. Convert a price stored as text into a numeric value and calculate tax.
10. Build a small product record using multiple data types and calculate its total value.

Strings

1. Clean ``rahul kumar`` by removing extra spaces and converting it to title case.
2. Create a full customer name from separate first-name and last-name variables.
3. Extract the first five characters from ``PythonDataAnalysis``.
4. Calculate the character count for each product name in a list.
5. Convert a list of customer names to uppercase.
6. Replace ``Mumbai`` with ``Delhi`` in a customer address.
7. Find customer names containing ``Kumar``.
8. Create an f-string showing customer name, order ID, and order amount.
9. Split ``Laptop,Electronics,45000`` into fields and convert the price to a number.
10. Clean a list of category names by stripping spaces and standardizing case.

Numbers & Arithmetic

1. Calculate total revenue from quantities ``[2,5,3,8]`` and prices ``[100,200,150,50]``.

2. Calculate a 15% discount on a product priced at 4800.
3. Calculate 18% tax on a bill of 2500 and the final amount.
4. Calculate percentage growth from 80000 to 95000.
5. Calculate the average of `[1200,850,2400,1750,990]`.
6. Calculate profit when revenue is 250000 and cost is 185000.
7. Calculate profit margin as a percentage.
8. Round an average order value to two decimal places.
9. Calculate the number of complete boxes when 237 units are packed 12 per box.
10. Calculate monthly take-home salary from salary, bonus, tax, and deductions.

Booleans & Comparisons

1. Create a Boolean checking whether a transaction is greater than 10000.
2. Create a Boolean checking whether a customer's age is between 18 and 60.
3. Check whether an order status equals `Delivered`.
4. Compare two product prices and print the result.
5. Check whether an order is below 500 or above 5000.
6. Check whether a customer belongs to `Gold` or `Platinum`.
7. Check whether a transaction is both completed and above 10000.
8. Create a Boolean indicating whether a product is in stock.
9. Filter a list of sales values using a condition for values above 5000.
10. Create Boolean flags for high-value, profitable, and repeat-customer records.

if / elif / else

1. Label an order `High`, `Medium`, or `Low` based on its amount.
2. Create a discount system with 20% above 10000, 10% above 5000, otherwise 0%.
3. Classify performance scores into Poor, Average, Good, or Excellent.
4. Calculate shipping cost based on order value.
5. Classify customers as New, Regular, or Loyal from order count.
6. Process a list of sales and label each value as above or below target.
7. Flag a transaction as suspicious when amount exceeds 50000.
8. Assign a product category based on price range.
9. Calculate tax using three different income ranges.
10. Process five transactions and print an action for each based on transaction amount.

for Loops

1. Loop through 10 sales values and print each value.
2. Calculate total sales from a list using a `for` loop.
3. Calculate the average of a list of sales using a loop.
4. Count how many transactions exceed 10000.
5. Print even order IDs from 1 through 20.

6. Create a list containing the square of every sales value.
7. Loop through customer names and print each name with its order count.
8. Find the largest sales value without using ``max()``.
9. Calculate transaction revenue from quantity and price lists.
10. Loop through five products and print ``product - price`` for each.

while Loops

1. Use a while loop to print order numbers from 1 to 10.
2. Keep adding daily sales until cumulative sales reach 100000.
3. Reduce inventory after each sale until stock reaches zero.
4. Keep asking for a positive sales value until a valid value is entered.
5. Double a marketing budget until it exceeds 100000.
6. Process transactions until a transaction value of 0 is encountered.
7. Calculate compound growth until an investment exceeds 100000.
8. Count increments required for a score to reach 100.
9. Apply a 10% discount repeatedly until price falls below 1000.
10. Build a simple data-entry loop that ends when the user enters ``quit``.

Functions

1. Write a function that calculates revenue from quantity and unit price.
2. Write a function that calculates profit from revenue and cost.
3. Write a function that calculates profit margin.
4. Write a function that calculates discount amount.
5. Write a function that classifies a transaction as High, Medium, or Low.
6. Write a function that returns the average of a list of sales.
7. Write a function that cleans a customer name.
8. Write a function that calculates average order value.
9. Write a function that returns the highest price from a list.
10. Write a function that accepts customer data and returns a formatted summary.

Error Handling

1. Ask for quantity input and handle non-numeric input.
2. Calculate profit margin and handle zero revenue.
3. Open a CSV file and handle a missing-file error.
4. Convert several entered prices to numbers and handle invalid entries.
5. Process transactions and skip records containing invalid numeric values.
6. Create a safe division function that handles division by zero.
7. Read a text file and handle a missing-file exception.
8. Build data entry that continues after an invalid transaction.
9. Handle invalid date input and request another value.

10. Create a calculation script with separate handling for invalid input and unexpected errors.

2. Python Data Structures

Lists

1. Create a list of 10 sales values and calculate its total.
2. Add a new transaction to a sales list.
3. Remove a cancelled order amount from a list.
4. Sort purchase amounts from highest to lowest.
5. Extract the first five sales values.
6. Replace one product name in a product list.
7. Count how many times a category occurs in a list.
8. Find highest and lowest sales values.
9. Create a list containing only sales above 5000.
10. Create a list of customer dictionaries and print customers spending above 10000.

Tuples

1. Create a tuple containing order ID, customer ID, and amount, then unpack it.
2. Store five monthly sales values in a tuple and calculate total sales.
3. Access the first and last month from a sales tuple.
4. Convert a transaction list into a tuple.
5. Convert a product tuple to a list and update the price.
6. Create a tuple of column names and use it to build a record.
7. Count occurrences of a value in a transaction tuple.
8. Find the index of a customer ID in a tuple.
9. Create a product tuple and unpack its fields.
10. Create a list of order tuples and print each order.

Dictionaries

1. Create a customer dictionary with ID, name, city, and spend.
2. Update a customer's spending value.
3. Add a `segment` field to a customer dictionary.
4. Create five product dictionaries and print each price.
5. Create customer dictionaries and calculate total spending.
6. Find the highest-spending customer.
7. Count customers by city using a dictionary.
8. Create a product-price dictionary and calculate a shopping cart total.
9. Check whether a customer ID exists before accessing it.
10. Convert order records into a dictionary keyed by order ID.

Sets

1. Create a set of customer IDs from a list containing duplicates.

2. Find customers who purchased in both January and February.
3. Find January customers who did not purchase in February.
4. Combine customer IDs from two campaigns.
5. Find unique product categories from a list.
6. Compare two employee-ID sets and find IDs missing from one.
7. Remove duplicate email addresses using a set.
8. Find common customers between two sales datasets.
9. Find unique cities across two customer lists.
10. Identify customers new in February compared with January.

Comprehensions

1. Create squares for numbers 1 through 20 using a list comprehension.
2. Create a list containing only sales above 5000.
3. Convert five customer names to uppercase.
4. Create discounted prices using a comprehension.
5. Create even transaction IDs from 1 through 50.
6. Create a product-to-price dictionary using a dictionary comprehension.
7. Create a customer-ID-to-order-count dictionary.
8. Create a cleaned category list using a comprehension.
9. Create a list containing only positive profit values.
10. Create High/Low transaction labels using a comprehension.

3. NumPy

Arrays

1. Create a NumPy array containing 10 daily sales values.
2. Create a 3x4 matrix representing three regions across four months.
3. Print the shape of a sales matrix.
4. Print dimensions and number of elements.
5. Create an array containing numbers from 1 to 100.
6. Create an array of 10 zeros.
7. Create an array of 10 ones.
8. Create a 3x3 matrix of sales values.
9. Access a specific row and column.
10. Create a NumPy array from a Python transaction list.

Indexing & Slicing

1. Extract the first five sales values.
2. Extract the last three transaction values.
3. Select every second value.
4. Extract the second row from a regional matrix.
5. Extract the third column from a monthly matrix.
6. Extract a 2x2 section from a matrix.
7. Select values greater than 10000 using Boolean indexing.
8. Select values between 1000 and 5000.
9. Replace negative values with zero using indexing.
10. Extract months 2 through 4 for one region.

Mathematical Operations

1. Increase every sales value by 10%.
2. Apply a 15% discount to product prices.
3. Calculate profit from revenue and cost arrays.
4. Calculate percentage growth between two sales arrays.
5. Multiply quantity and price arrays to calculate revenue.
6. Calculate square roots of an array.
7. Calculate current-month minus previous-month sales.
8. Normalize an array by its maximum value.
9. Calculate absolute changes in an array.
10. Calculate profit margins using vectorized operations.

Aggregations

1. Calculate total sales from an array.

2. Calculate average daily sales.
3. Find minimum and maximum transaction values.
4. Calculate median order value.
5. Calculate standard deviation of daily sales.
6. Calculate total sales for each region in a 2D array.
7. Calculate average sales for each month.
8. Find the region with the highest total sales.
9. Calculate mean and maximum for each matrix column.
10. Calculate multiple summary statistics for transactions.

4. Pandas

Series & DataFrames

1. Create a Series from five daily sales values.
2. Create a DataFrame from customer IDs, names, and purchases.
3. Display the first five rows.
4. Display the last five rows.
5. Print row and column counts.
6. Print all column names.
7. Print all data types.
8. Select one column and calculate its average.
9. Select two columns and display them.
10. Create a DataFrame from a list of order dictionaries.

Reading CSV / Excel

1. Read `sales.csv` into a DataFrame and display the first five rows.
2. Read `sales.xlsx` and inspect its columns.
3. Load a specific worksheet from an Excel workbook.
4. Write a cleaned DataFrame to CSV.
5. Write a summary DataFrame to Excel.
6. Read a CSV using a non-default delimiter.
7. Load a CSV and print shape, columns, and types.
8. Load a dataset and identify missing-value columns.
9. Load two CSV files and compare their columns.
10. Load a file, validate required columns, and stop if a required column is missing.

Selecting & Filtering

1. Select the `sales` column.
2. Select `customer`, `region`, and `sales`.
3. Filter transactions where sales exceed 10000.
4. Filter North-region orders with sales above 5000.
5. Filter products whose category is in three selected categories.
6. Filter customer names containing `an`.
7. Use `loc` to select rows by labels.
8. Use `iloc` to select rows by position.
9. Filter transactions after a specified date.
10. Create a DataFrame containing only high-value completed orders.

Sorting

1. Sort orders by sales descending.

2. Sort customers by region and then sales.
3. Sort products by price ascending.
4. Display the top 10 sales records.
5. Display the bottom 10 records.
6. Sort two columns using different directions.
7. Sort customers by order count and average order value.
8. Reset the index after sorting.
9. Find the highest-selling product after sorting.
10. Rank regions by total sales.

groupby

1. Group sales by region and calculate total revenue.
2. Group orders by product and calculate average order value.
3. Count orders by customer segment.
4. Calculate total, average, minimum, and maximum sales by region.
5. Group by region and product and calculate revenue.
6. Find the region with the highest total sales.
7. Find the product with the highest average order value.
8. Calculate monthly revenue totals.
9. Calculate unique customers per region.
10. Create a KPI summary by region containing sales, orders, and AOV.

merge / joins

1. Merge customers and orders using `customer\_id`.
2. Perform a left join and identify customers with no orders.
3. Perform an inner join and count resulting rows.
4. Merge products with order details using `product\_id`.
5. Merge DataFrames whose key columns have different names.
6. Check duplicate customer IDs before merging.
7. Compare row counts before and after merging.
8. Identify orders missing product information after a left join.
9. Create a many-to-one customer-to-region merge.
10. Build a customer-orders analysis using Pandas merge operations.

concat

1. Concatenate January and February sales vertically.
2. Combine three regional DataFrames.
3. Concatenate two DataFrames with matching columns.
4. Concatenate DataFrames with different columns and inspect missing values.
5. Combine monthly sales files into one DataFrame.

6. Concatenate two DataFrames horizontally.
7. Reset the index after combining datasets.
8. Add a month column to monthly DataFrames before concatenating.
9. Combine four quarterly DataFrames into an annual DataFrame.
10. Create a reusable process that concatenates a list of monthly DataFrames.

Missing Values

1. Count missing values in every column.
2. Calculate missing-value percentage by column.
3. Drop rows where customer ID is missing.
4. Fill missing sales with zero and compare totals.
5. Fill missing categories with `Unknown`.
6. Fill missing numeric values using the median.
7. Identify columns with more than 20% missing values.
8. Create a missing-value report with counts and percentages.
9. Compare mean and median imputation on a numeric column.
10. Create a cleaned DataFrame while preserving the original.

Duplicates

1. Find duplicate rows.
2. Count duplicate customer IDs.
3. Remove completely duplicated rows.
4. Remove duplicate orders based on order ID.
5. Keep the latest record for duplicate customer IDs.
6. List duplicates before deleting them.
7. Compare row counts before and after duplicate removal.
8. Calculate whether duplicate orders inflate revenue.
9. Remove duplicate product records by product ID.
10. Create a validation check for unexpected duplicate keys.

Data Types

1. Display all DataFrame data types.
2. Convert quantity from string to integer.
3. Convert price from string to numeric.
4. Convert order date to datetime.
5. Handle invalid numeric values during conversion.
6. Convert a category column to categorical type.
7. Identify numeric columns incorrectly stored as text.
8. Extract year and month after converting dates.
9. Create a required-column data-type validation report.

10. Fix mixed numeric and text values in a sales column.

Dates & Times

1. Convert an order-date column to datetime.
2. Create year, month, and day columns.
3. Calculate delivery time in days.
4. Filter orders from January 2026.
5. Calculate monthly revenue.
6. Calculate daily order counts.
7. Identify orders delivered more than seven days after purchase.
8. Calculate days since each customer's last order.
9. Group sales by weekday.
10. Create a chronologically sorted monthly sales DataFrame.

Calculated Columns

1. Create revenue from quantity multiplied by unit price.
2. Create profit from revenue minus cost.
3. Create profit-margin percentage.
4. Create a Boolean column for orders above 10000.
5. Create discounted price from price and discount percentage.
6. Create customer-level average order value.
7. Create customer segments from total spending.
8. Create a delivery-status flag from delivery days.
9. Create a month-year column from transaction dates.
10. Create at least three business metrics from a sales DataFrame.

5. Data Visualization

Matplotlib

1. Create a line chart of monthly revenue.
2. Create a bar chart comparing revenue across five regions.
3. Create a histogram of order values.
4. Create a scatter plot of advertising spend versus revenue.
5. Create a chart with a title and labeled axes for monthly sales.
6. Create a chart comparing two product categories with a legend.
7. Create a dashboard-ready chart with a specified figure size.
8. Save a sales chart as PNG.
9. Create a line chart comparing two years of monthly sales.
10. Create a bar chart of the top 10 products by revenue.

Seaborn

1. Create a Seaborn bar chart of sales by region.
2. Create a histogram of transaction amounts.
3. Create a box plot of order values by customer segment.
4. Create a scatter plot of price versus quantity.
5. Create a count plot of orders by category.
6. Create a heatmap from a correlation matrix.
7. Create a box plot comparing sales distributions across regions.
8. Create a chart showing monthly sales.
9. Create a categorical visualization of product sales.
10. Create three Seaborn charts covering sales, orders, and profit.

Chart Selection

1. Create a chart that shows monthly sales trends.
2. Create a chart comparing sales across 10 regions.
3. Create a chart showing the distribution of order values.
4. Create a chart showing the relationship between price and quantity.
5. Create a chart comparing order-value distributions across customer segments.
6. Create a chart highlighting the top 10 products by revenue.
7. Create a chart comparing conversion rates across marketing channels.
8. Create a chart making unusual daily-order spikes visible.
9. Create a chart comparing two years of monthly revenue.
10. Create three candidate charts for one dataset and choose the clearest one by comparing them.

Data Storytelling

1. Create a chart highlighting the month with the largest sales decline.

2. Create a chart highlighting the top-performing region.
3. Annotate a chart at the month of peak revenue.
4. Create KPI visualizations for revenue, orders, and profit.
5. Create a chart comparing actual sales against target.
6. Create a visualization showing a product's share of total revenue.
7. Create a before-and-after campaign sales visualization.
8. Create a chart emphasizing an unusually high return rate.
9. Build three charts in an order that communicates a business story.
10. Create a visualization and produce three quantified findings from it.

6. Exploratory Data Analysis (EDA)

Dataset Understanding

1. Load a dataset and print shape, columns, types, and first five rows.
2. Calculate unique-value counts for every column.
3. Identify columns containing missing values.
4. Identify numeric and categorical columns programmatically.
5. Calculate summary statistics for numeric columns.
6. Identify likely ID columns by checking duplicate counts.
7. Inspect key columns to determine the dataset's transaction grain.
8. Create a data-quality summary of rows, columns, missing values, and duplicates.
9. List the top five values and frequencies for every categorical column.
10. Create an initial EDA report that flags columns requiring investigation.

Outliers

1. Create a box plot for order values and identify potential outliers.
2. Calculate Q1, Q3, and IQR for sales.
3. Return rows whose sales fall outside IQR-based bounds.
4. Compare mean and median before and after excluding extreme values.
5. List the 10 largest transactions for investigation.
6. Identify products with unusually high or low average order values.
7. Compare regional sales distributions with box plots.
8. Flag transactions above a chosen business threshold.
9. Compare how extreme values affect total and average revenue.
10. Create an outlier-review table containing transaction, amount, customer, and product fields.

Relationships & Correlation

1. Calculate a correlation matrix for numeric columns.
2. Create a heatmap of the correlation matrix.
3. Create a scatter plot of marketing spend versus revenue.
4. Calculate correlation between price and quantity.
5. Calculate correlation between discount and order value.
6. Find the pair with the strongest absolute correlation.
7. Compare correlation before and after removing data errors.
8. Create scatter plots for three selected variable pairs.
9. Calculate correlations separately by region.
10. Use the correlation results to select three relationships for deeper analysis.

EDA Insights

1. Find the highest-revenue region and calculate its share of total revenue.

2. Find the month with the largest revenue decline.
3. Find the top five products by profit.
4. Find the customer segment with the highest average order value.
5. Find products with high revenue but low profit margin.
6. Identify customers who have not ordered recently.
7. Find categories with unusually high return rates.
8. Compare average order value across regions and segments.
9. Find one major business pattern and support it with at least three calculated metrics.
10. Create a one-page EDA summary containing five quantified findings.

7. Real Data Analyst Projects

Sales Analysis Project

1. Load a sales dataset and calculate revenue, cost, profit, and margin.
2. Create a monthly revenue trend.
3. Rank products by revenue and profit.
4. Compare sales performance across regions.
5. Calculate average order value by month.
6. Find the top 10 customers by revenue.
7. Find months where revenue declined from the previous month.
8. Identify products with high revenue but low profit.
9. Create three visualizations explaining the main sales patterns.
10. Export a cleaned dataset and KPI summary.

Customer Analysis Project

1. Calculate total revenue per customer.
2. Rank customers by total spending.
3. Calculate order count per customer.
4. Calculate average order value per customer.
5. Create customer segments based on spending.
6. Identify customers with only one order.
7. Identify customers whose latest order is older than a chosen threshold.
8. Compare spending across customer segments.
9. Create charts showing customer distribution and revenue contribution.
10. Build a customer-analysis table containing ID, orders, revenue, AOV, segment, and last-order date.

E-commerce Analysis Project

1. Calculate total orders, revenue, profit, and AOV.
2. Rank products by revenue.
3. Calculate category-level revenue and profit.
4. Calculate daily and monthly order trends.
5. Compare performance across acquisition channels.
6. Calculate return rate by category.
7. Identify high-value customers.
8. Find products with high sales and high return rates.
9. Create visualizations covering revenue, products, customers, and returns.
10. Create a final EDA dataset and KPI summary suitable for Power BI.

Marketing Analysis Project

1. Calculate campaign spend, clicks, conversions, and revenue.

2. Calculate click-through rate by campaign.
3. Calculate conversion rate by channel.
4. Calculate cost per acquisition.
5. Calculate return on ad spend.
6. Rank campaigns by revenue and ROAS.
7. Compare marketing performance by channel.
8. Create a monthly campaign-performance trend.
9. Identify channels with high spend and low conversion.
10. Create a final campaign KPI summary table.

HR / Employee Analysis Project

1. Calculate employee count by department.
2. Calculate average salary by department.
3. Calculate employee turnover rate.
4. Calculate average tenure by department.
5. Compare turnover across departments.
6. Create a salary-distribution histogram or box plot.
7. Identify departments with unusually high attrition.
8. Compare attrition by tenure band.
9. Create a department KPI table with headcount, salary, tenure, and attrition.
10. Create three visualizations explaining the main workforce patterns.

8. Interview Preparation

Python Interview Practice

1. Write a function that calculates profit margin and test it with three inputs.
2. Remove duplicates from a list while preserving original order.
3. Find the second-highest sales value in a list.
4. Count the frequency of product categories in a list.
5. Return the top three values from a list.
6. Combine two customer-ID lists and return unique IDs.
7. Find customer IDs present in one list but not another.
8. Convert transaction dictionaries into a summary dictionary.
9. Clean a list of customer names and remove blank entries.
10. Write a Python program that loads transactions, calculates KPIs, and prints a summary.

Pandas Interview Practice

1. Filter orders above 10000 and sort descending.
2. Merge customers and orders and calculate revenue by customer.
3. Find duplicate order IDs and return affected rows.
4. Calculate total and average sales by region.
5. Find the top five products by revenue.
6. Calculate monthly revenue from transaction dates.
7. Find columns with more than 10% missing values.
8. Create profit margin and identify products below a target margin.
9. Find customers with no order in the last 90 days.
10. Perform an end-to-end Pandas task: clean, join, aggregate, and visualize a sales dataset.

Case Study Practice

1. Given monthly revenue with a 15% decline, write code identifying regions causing the decline.
2. Given product revenue and cost, identify products with increased revenue but decreased profit.
3. Given customer history, identify customers whose order frequency has fallen.
4. Given marketing spend and conversion data, calculate the worst cost-per-acquisition channel.
5. Given transactions, identify unusual spikes in transaction value.
6. Given two sales datasets, compare totals and identify records causing differences.
7. Given missing customer IDs, quantify missingness and revenue affected.
8. Given orders and returns, calculate return rate by product.
9. Given regional sales, rank regions and calculate the largest performance gap.
10. Given a raw CSV, complete cleaning, KPI calculation, EDA, visualization, and export of a final summary.